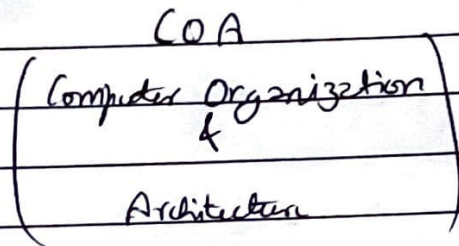


Name: Mandeep Singh
Roll No.: 2301420049
Programme: B.Tech CSE (Data Science)

DATE



Assignment - 4

(Section - A)

(i) SRAM v/s DRAM
(Static RAM) & (Dynamic RAM)

- SRAM = uses flip-flops to store data, making it faster, more reliable & not requiring refreshing. However, it is expensive & consumes more space, so it is used in cache memory.

- DRAM = uses capacitors to store charges, which leaks over time, requiring periodic refreshing. It is slower but cheaper & denser, making it suitable for main memory.

(ii) Locality of Reference:

Locality of reference is the principle that programs tend to access a small portion of memory repeatedly.

- ~~Temporary~~ Temporal Locality: recently accessed data is likely to be accessed again.

- Spatial Locality: Nearby memory locations are likely to be ~~access~~ accessed.

↳ This principle is the foundation of cache memory design.

Spiral

(iii) Cache Memory:

Cache memory is small, high-speed memory located b/w CPU & main memory.

It stores frequently accessed data & instructions, reducing average memory access time & improving overall system performance.

(iv) Cache Indexing:

Cache indexing determines how many memory blocks are mapped to ~~cache~~ cache locations:

- Direct mapped: Each block maps to one fixed location
- Set associative: Block maps to a set of locations
- Fully associative: Block can be placed anywhere.

(v) Cache Block Size:

Cache block size affects performance:

- Larger blocks improve spatial locality
- But ~~may~~ increase miss penalty & may bring unnecessary data (cache pollution).

(vi) Cache Associativity

- Cache associativity defines how many locations a memory block can occupy.
- Higher associativity ~~reduces~~ reduces conflict misses but increases complexity & access time.

(vii) Cache Size Tradeoff

- Larger cache:
 - Reduces miss rate
 - Improves Performance
- But,
 - Increases Cost
 - Increases access latency

DATE

(iv) Write Policies:

- Write-through: data is written to both cache & main memory simultaneously $\rightarrow$ simpler but $\uparrow$ ~~rate~~ memory traffic
- Write-back: Data is written only to cache & updated in memory later $\rightarrow$ faster but more complex

(ix) ~~(viii)~~ AMAT (Avg Memory Access Time)

$$\text{AMAT} = \underbrace{\text{Hit Time}}_{\substack{\downarrow \\ \text{Time to access} \\ \text{Cache}}} + \underbrace{(\text{Miss Rate} \times \text{Miss Penalty})}_{\substack{\downarrow \\ \text{fraction of} \\ \text{accesses that} \\ \text{miss}}} \underbrace{}_{\substack{\downarrow \\ \text{Time to fetch} \\ \text{from main} \\ \text{memory}}}$$

(x) LRU Replacement Policy

Least Recently Used replaces the block that has ~~be~~ not been used for the longest time, improving cache efficiency

(xi) Memory Interleaving

Memory is divided into ~~the~~ multiple modules that can be accessed simultaneously, increasing bandwidth & reducing delay

(xii) Magnetic Disk

- Tracks = circular paths on disk
- Sectors = subdivisions of tracks storing data

DATE

(XIII) Flash Memory

- Non volatile memory used in SSDs & USB drives
- Faster than magnetic disks & has no moving part, making it more reliable.

(XIV) Memory-Mapped I/O

- I/O devices ~~that~~ share the same address space as memory & are accessed using normal instructions.

(XV) Programmed I/O

- CPU continuously checks device status & waits → inefficient but simple

(XVI) I/O-Mapped I/O

- ~~separate~~ Separate address space for I/O devices, accessed using special instructions.

(XVII) Interrupt-driven I/O

Device interrupts CPU when ready → better CPU utilization

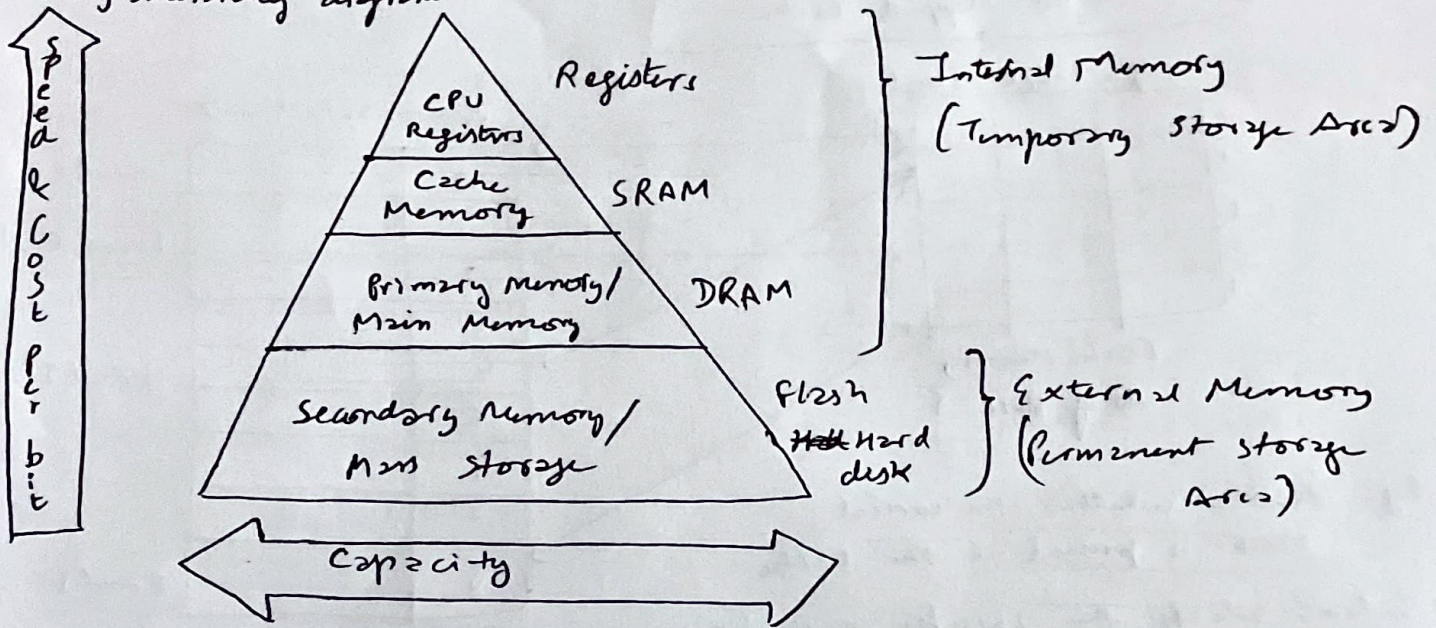
(XVIII) DMA

Direct Memory Access allows devices to transfer data directly to memory, without CPU involvement → highly efficient for large data transfers.

(Section - B)

(Diagram Based)

(1) Memory hierarchy diagram



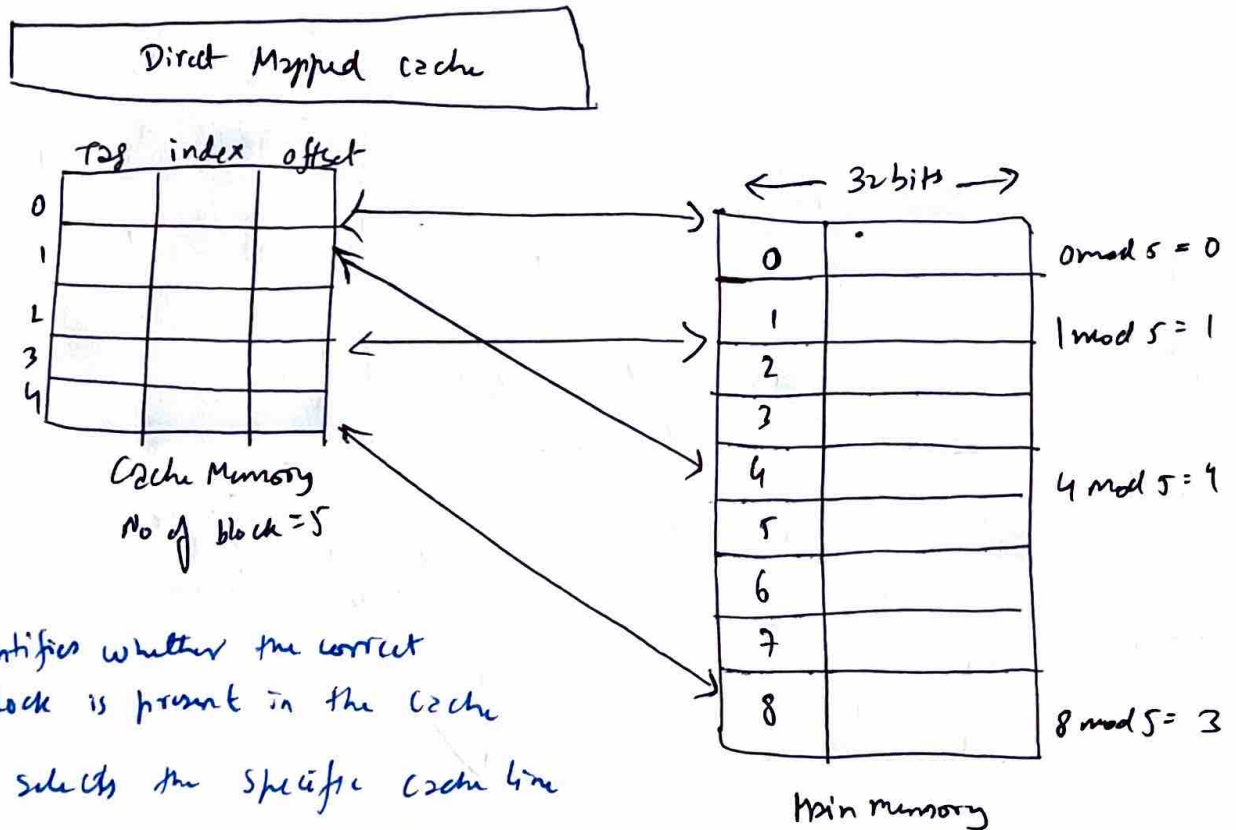
• Memory Hierarchy is an arrangement of different types of memory in a computer system based on speed, cost, size. It is designed to provide a balance b/w high performance & low cost.

- (i) Registers: Located inside CPU, they are the fastest & smallest memory units used to store & immediate data & instructions during execution.
- (ii) Cache memory: small, high-speed memory placed b/w CPU & main memory. It stores frequently used data to reduce access time.
- (iii) Main Memory (RAM): stores programs & data currently in use. It is slower than cache but larger in size.
- (iv) Secondary storage: includes hard disks & SSDs. It is non-volatile, has large capacity, but it is much slower compared to primary memory.

(17)

(11) Direct map cache organization

Diagram



- **Tag**: identifies whether the correct block is present in the cache
- **Index**: selects the specific cache line
- **Offset**: selects the exact data within the block

working: (i) CPU generates 2 memory addresses

(ii) Index is used to locate the cache line

(iii) Tag is compared with stored tag

• If match $\rightarrow$ Cache Hit

• If not $\rightarrow$ Cache Miss

(iv) on miss, block is fetched from the main memory & placed in that fixed line.

* Direct Mapping provides fast access but suffers from conflict misses due to fixed mapping of memory blocks.

(11) Write through v/s Write Back

H Diagram:

(1) Write Through:

	V	Tag	Data
100	1	10101	0303

MAIN MEMORY	
1010 1100	0303

After write through

	V	Tag	Data
100	1	10101	2203

MAIN MEMORY	
1010 1100	2203

(11) write Back:

	V	Tag	Data
100	1	10101	0303

Main Memory	
1010 1100	0303

After write Back

	V	Tag	Data
100	1	10101	2203

When Cache line is about to be replaced the data will be updated

Main Memory	
1010 1100	0303

	V	Tag	Data
100	1		SOME REPLACED DATA

Main Memory	
1010 1100	2203

• Write through Policy:

Whenever CPU writes data:

- Data is written ~~both~~ to both cache & main memory simultaneously
- Ensures memory always has latest data

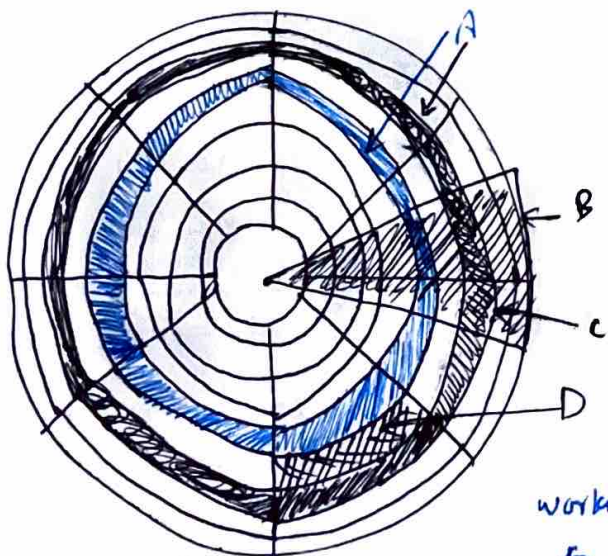
• Write Back Policy:

- Data written only to cache initially
- Main memory is updated later when the cache block is replaced

Diff:

feature	Write through	Write Back
Memory update	immediate	Delayed
speed	Slower	faster
Memory Traffic	High	Low
Complexity	Simple	Complex

(IV) Magnetic Disk Structure



(A) Track

(B) Geometrical sector

(C) Disk Sector

(D) Cluster

• track helps organize data in orderly manner

• sectors allow disk to locate & access data efficiently

working: when data is read or written: The disk rotates → read/write head moves to correct track → data is collected from required sector

(Section - I)

(I) Repeated Memory Access:

- Program repeatedly access the same memory location, this follows temporally locality, which means recently accessed data is likely to be accessed again in the near future.
- Since the same memory location is accessed repeatedly, the system benefits from keeping the data in cache, reducing access time & improving performance.

(II) Large Cache Block Size Issues

- Although larger block size improves spatial locality & reduces miss rate, it can degrade performance because:
 - Miss penalty $\uparrow$ = more data must be fetched from memory
 - Cache Pollution = unnecessary data occupies cache space
 - Fewer blocks fit in cache = increases conflict misses

Thus, overall performance may decrease despite lower miss rate.

(III) Write through Cache Operation

- In write-through cache:
 - Data is written to both cache & main memory simultaneously

This ensures data consistency b/w cache & memory but $\uparrow$ memory traffic & ~~reduces~~ reduces performance.

DATE

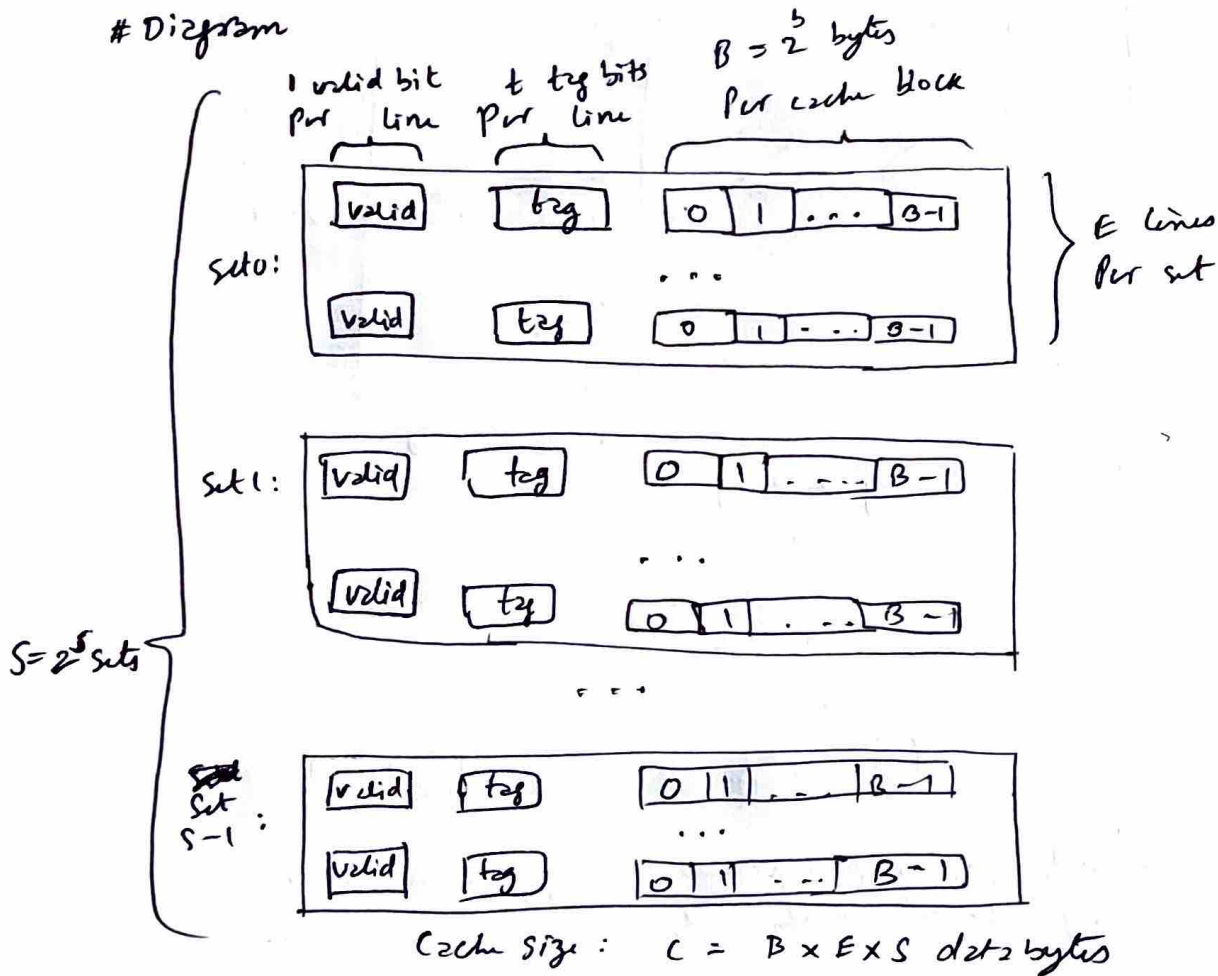
(IV) High-Speed Network Transfer

- High speed network card transfers large data blocks to memory without CPU involvement. This uses DMA (Direct Memory Access) allowing data transfer directly b/w I/O device & main memory without CPU intervention, improving efficiency & freeing the CPU for other tasks.

(Section - D)

(1) Cache Memory Organization

Diagram



Cache memory organization defines how data from main memory is stored & accessed in cache

Address mapping: Memory Address is divided into 3 parts:

- Tag: identifies the memory block
- Index: Selects a specific cache line
- Offset: Selects data within the block

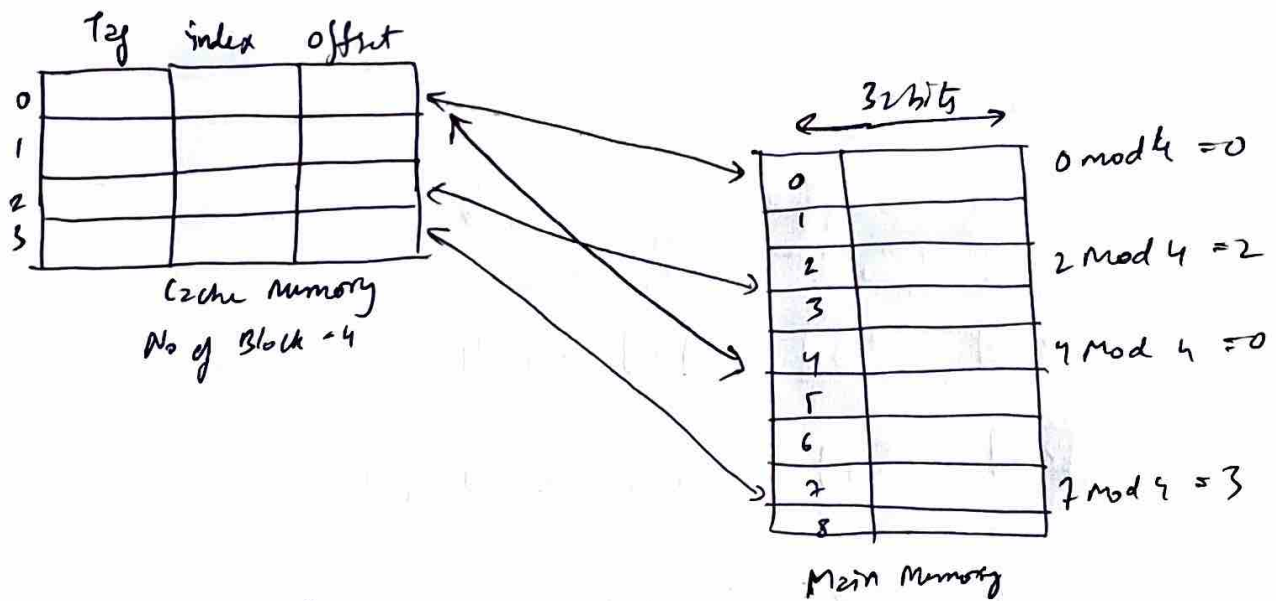
Working of Cache mapping: (i) CPU generates a memory address

(ii) Index selects cache line

- (iii) Tag of address is compared with stored tag
- If match $\rightarrow$ Cache hit
 - If not $\rightarrow$ Cache miss

(iv) On miss, block is loaded into cache

Diagram of Mapping:



(ii) AMAT eqⁿ & factors affecting memory performance

AMAT Eqⁿ:

$$AMAT = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Avg Memory Access Time (AMAT) represents avg time taken to access memory, considering both cache hits & cache misses.

Factors affecting Memory Performance

(i) Cache Size: • Larger cache → lower miss rate
• But may increase access time

(ii) Cache Block Size: • Larger Block → better spatial locality
• But ↑ miss penalty

(iii) Cache Associativity: • Higher Associativity → fewer conflict misses
• But increases complexity & latency

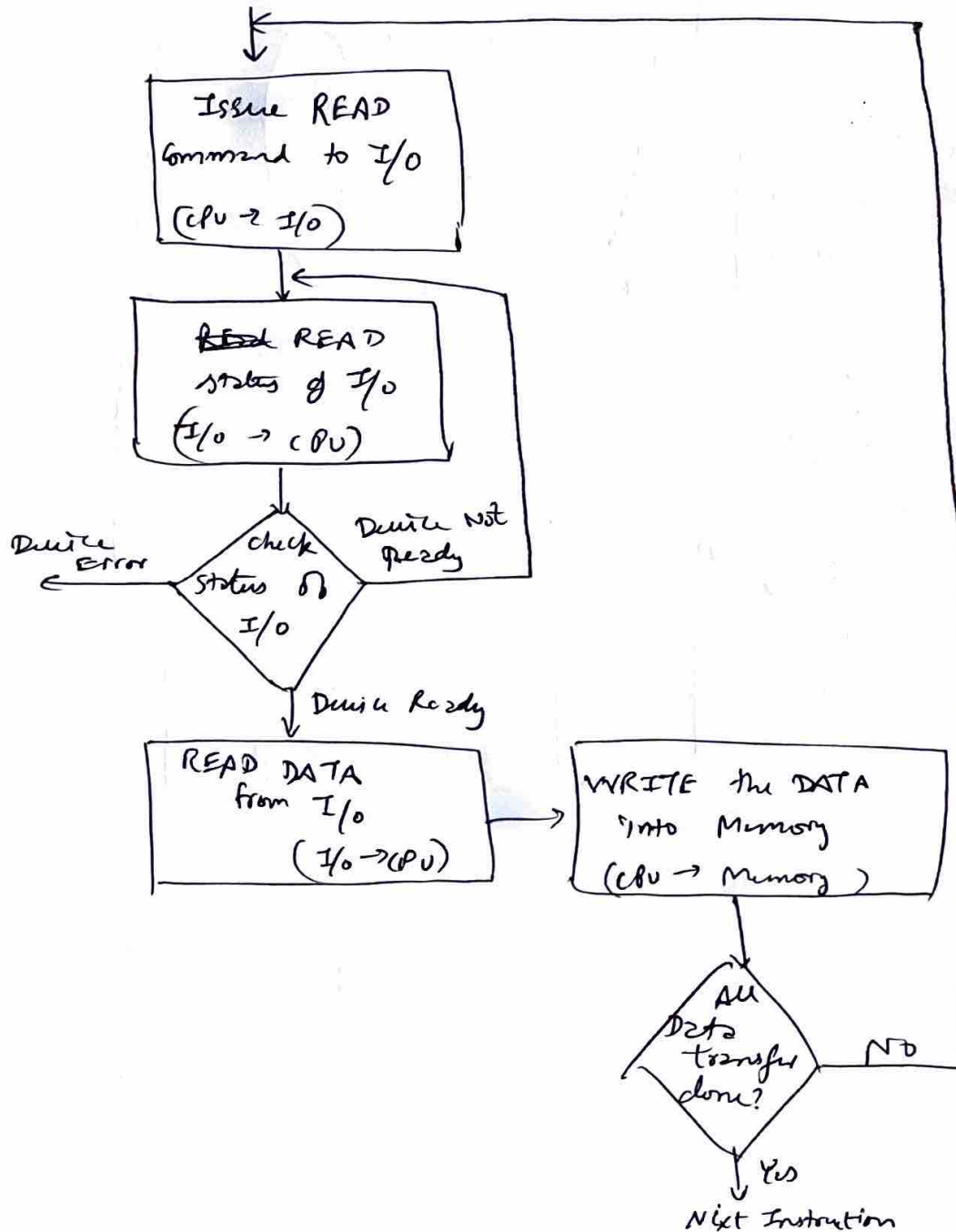
(iv) Replacement Policy: Efficient policies (like LRU) improve hit rate

(v) Write Policy: • Write back improves performance
• Write through increases memory traffic

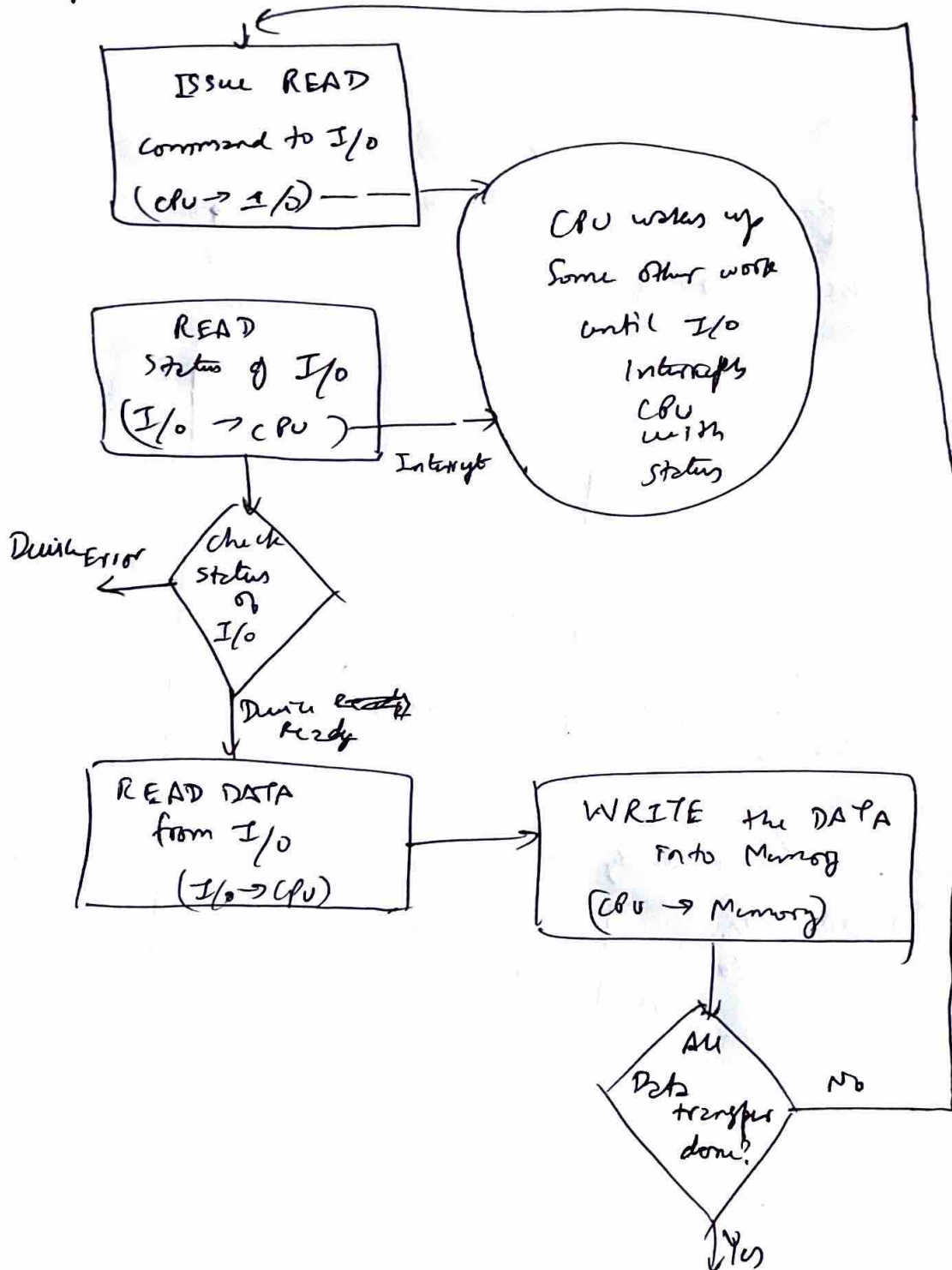
(11) Data I/o data transfer techniques:

Diagrams:

Programmed I/o Data Transfer



Interrupt Driven I/O Data Transfer



DMA: I/O Data Transfer:

CPU sends details to DMA

- (i) Device on which I/O to be carried out
- (ii) What is the command (R/W) to be carried out
- (iii) Starting address of memory location for Data Transfer
- (iv) Length of the Data Transfer

[CPU → DMA]

CPU freed
for other tasks

I/O operation delegated
to
DMA



DMA Data Transfer to
Memory

- (i) For Data Transfer DMA contends for the system bus
- (ii) Directly transfers to the said location in memory, for length of data defined by CPU [DMA → Memory]
- (iii) When Memory Data Transfer is complete, generates interrupt to CPU

[DMA → CPU]

CPU not disturbed until the interrupt is

generated. Interrupt is serviced by CPU

at its convenient point. DMA waits for interrupt service by CPU

Interrupt
to CPU

Advantages of Each I/O data transfer:

(i) Programmed I/O: Simple to implement

(ii) Interrupt Driven I/O: Better CPU utilization

(iii) DMA (Direct Memory Access): High speed data transfer with minimal CPU involvement

DMA is most efficient technique